

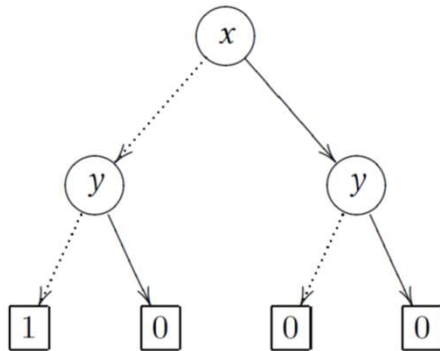
BDD

Representations of Boolean Functions

Representation	Compact?	Test for		Operations		
		Satisfiability	Validity	.	+	¬
Truth tables	Never	Hard	Hard	Hard	Hard	Hard
Prop. formulas	Often	Hard	Hard	Easy	Easy	Easy
DNF formulas	Sometimes	Easy	Hard	Hard	Easy	Hard
CNF formulas	Sometimes	Hard	Easy	Easy	Hard	Hard
ROBDD	Often	Easy	Easy	Medium	Medium	Easy

Note: With a truth table representation, while operations are conceptually easy, especially when table rows are always listed in some standard order, the time complexities are hard, as table sizes and, hence, operation time complexities are always exponential in the number of input variables.

Binary Decision Trees



$$f(x, y) = x'.y'$$

To compute value:

1. Start at root
2. Take dashed line if value of var at current node is 0
3. Take solid line if value of var at current node is 1
4. Function value is value at terminal node reached

Binary decision tree has $2^{n+1} - 1$ nodes for an n variable Boolean function – [not a compact representation](#)

Binary Decision Diagram

Similar to Binary Decision Trees, except that nodes can have multiple in-edges.

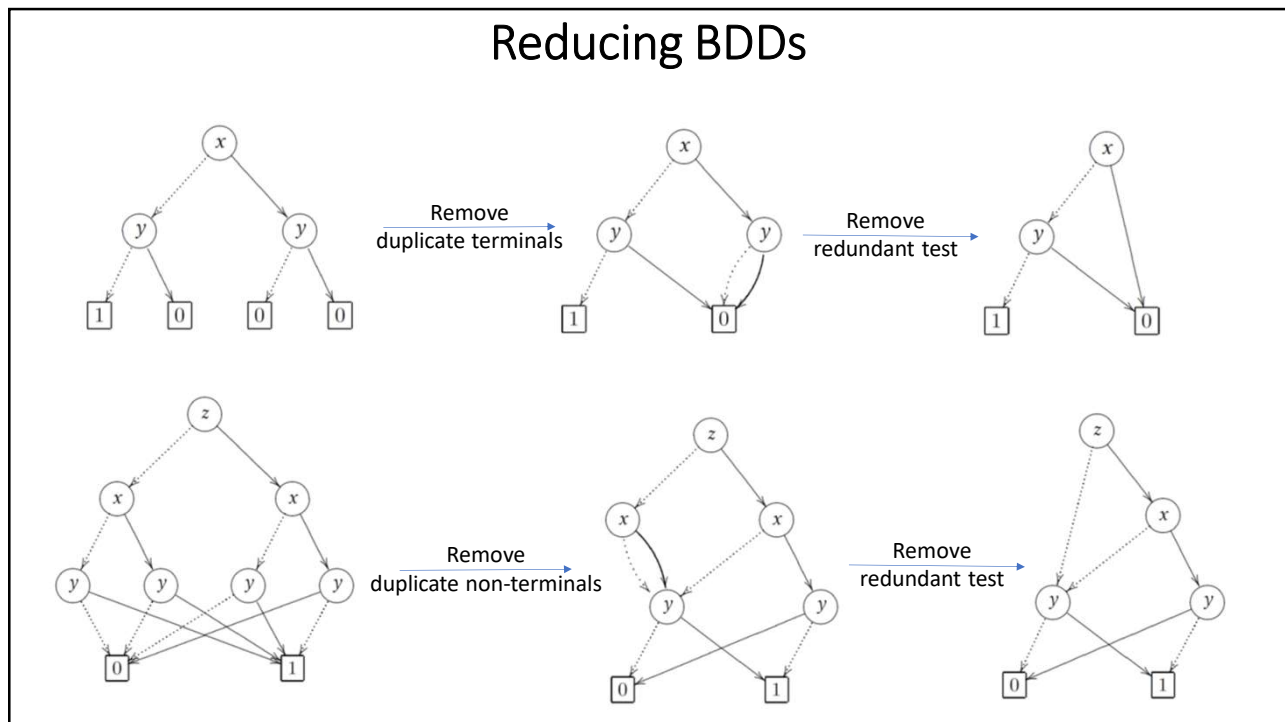
A **binary decision diagram** (BDD) is a finite DAG (Directed Acyclic Graph) with:

- a unique initial node;
- all non-terminals labelled with a Boolean variable;
- all terminals labelled with 0 or 1;
- all edges are labelled 0 (dashed) or 1 (solid);
- each non-terminal has exactly: one out-edge labelled 0, and one out-edge labelled 1.

We will use BDDs with two extra properties:

1. *Reduced* – eliminate redundancy
2. *Ordered* – canonical ordering of the Boolean variables

Reducing BDDs



BDD Reduction Operations

1. Removal of duplicate terminals. If a BDD contains more than one terminal 0-node, then redirect all edges that point to such a 0-node to just one of them. Do the same with terminal nodes labeled 1.
2. Removal of redundant tests. If both outgoing edges of a node n point to the same node m , then remove node n , sending all its incoming edges to m .
3. Removal of duplicate non-terminals. If two distinct nodes n and m in the BDD are the roots of structurally identical subBDDs, then eliminate one of them and redirect all its incoming edges to the other one.

Note: A BDD is said to be reduced if no more reductions are possible

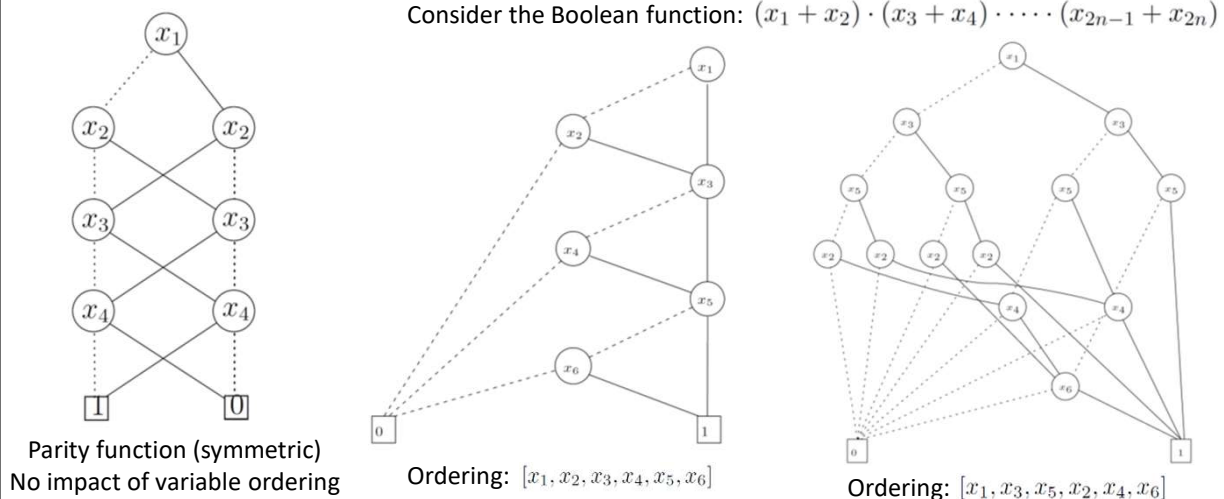
Ordered BDD

- Let $[x_1, x_2, \dots, x_n]$ be an ordered list of variables without duplicates;
- A BDD B has an ordering $[x_1, x_2, \dots, x_n]$ if
 - all variables of B occur in $[x_1, x_2, \dots, x_n]$ and
 - if x_j follows x_i on a path in B then $j > i$ i.e., **unique variable ordering is followed by every path**
- An **ordered BDD** (OBDD) is a BDD which has an ordering for some list of variables.
- The orderings of two OBDDs B and B' are **compatible** if there are no variables x, y such that
 - x is before y in the ordering for B , and
 - y is before x in the ordering for B'

Theorem[Canonicity]: For a given ordering, the reduced OBDD (ROBDD) representing a given function f is **unique**

If B_1 and B_2 are two ROBDDs with compatible variable orderings representing the same Boolean function, then they have identical structure

Impact of Variable Ordering



- Various heuristics can help to choose orderings. The problem of finding the best variable ordering is NP-hard.
- No efficient ROBDD representation for multiply operation (they are all exponential size in the number of Boolean variables).

Importance of Canonical Representation

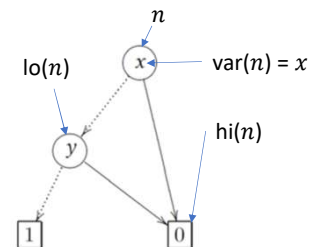
Canonical representation enables easy test for:

- **Absence of redundant variables:** A Boolean function f does not depend on an input variable x if no nodes occur for x in the ROBDD for f .
- **Semantic equivalence:** Check $f \equiv g$ by checking whether or not the ROBDDs for f and g have identical structures.
- **Validity:** Check if the BDD is identical to the one with just the terminal node **1** and nothing else.
- **Satisfiability:** Check if the BDD is not identical to the one with just the terminal node **0** and nothing else.
- **Implication:** Check if $\forall \vec{x}. f(\vec{x}) \rightarrow g(\vec{x})$ by checking whether or not the ROBDDs for $f + \neg g$ is constant 0.

Operations on BDDs: Reduce

Some notation, for a given BDD node n :

- If n is a non-terminal node:
 - **var(n)** — the variable label on node n ;
 - **lo(n)** — the node reached by following the 0 edge from n ;
 - **hi(n)** — the node reached by following the 1 edge from n ;
- If n is a terminal node:
 - **val(n)** — the truth value labeling n



Reduce constructs a ROBDD from an OBDD

1. Label each OBDD node n with an integer $id(n)$,
2. in a single bottom-up pass, such that:
3. two OBDD nodes m and n have the same label ($id(m) = id(n)$) if and only if m and n represent the same Boolean function.

The ROBDD is then created by using one node from each class of nodes with the same label

Operations on BDDs: Reduce

Rules for assigning label to a node n :

Remove duplicate terminals:

if n is a terminal node (i.e., 0 or 1), then set $\text{id}(n)$ to be $\text{val}(n)$.

Remove redundant tests:

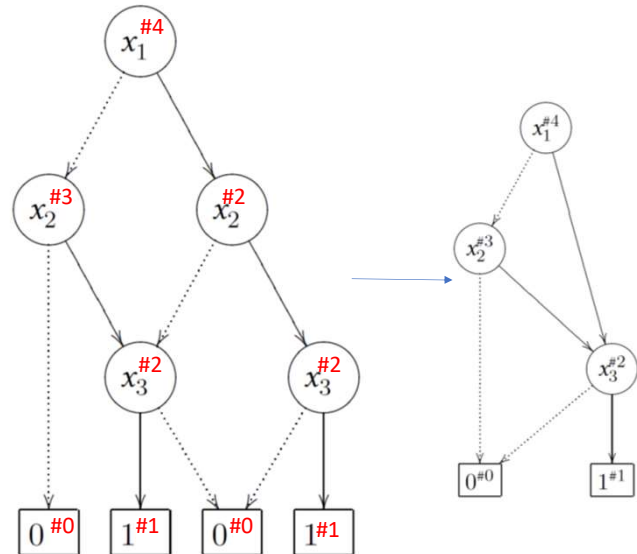
if $\text{id}(\text{lo}(n)) = \text{id}(\text{hi}(n))$ then set $\text{id}(n)$ to be $\text{id}(\text{lo}(n))$.

Remove duplicate nodes:

if there exists a node m that has already been labelled such that

$$\left[\begin{array}{l} \text{var}(m) = \text{var}(n) \\ \text{lo}(m) = \text{lo}(n) \\ \text{hi}(m) = \text{hi}(n) \end{array} \right] \text{ set } \text{id}(n) \text{ to } \text{id}(m).$$

Use a hashtable with $\langle \text{var}(n), \text{lo}(n), \text{hi}(n) \rangle$ keys for $O(1)$ lookup time.



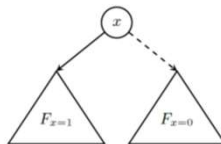
Operations on BDDs: Apply

Given compatible OBDDs B_f and B_g that represent formulas f and g ,

$\text{apply}(\square, B_f, B_g)$ computes an OBDD representing $f \square g$.

- where $\square$ represents some binary operation on Boolean formulas (e.g., $+$, $\cdot$, $\oplus$)
- Unary operations can be handled too (e.g., $\neg x = x \oplus 1$)

Shannon expansion: Any Boolean function can be written as $F \equiv (\neg x F_{x=0} + x F_{x=1})$



Apply: $f \square g$ can be expanded as $f \square g = \neg x (f_{x=0} \square g_{x=0}) + x (f_{x=1} \square g_{x=1})$

Remark: We can implement “apply” recursively on the structure of BDDs

Operations on BDDs: Apply

$$\text{apply}(\square, \begin{array}{c} \textcircled{x} \\ \swarrow \quad \searrow \\ B \quad B' \end{array}, \begin{array}{c} \textcircled{x} \\ \swarrow \quad \searrow \\ C \quad C' \end{array}) = \begin{array}{c} \textcircled{x} \\ \swarrow \quad \searrow \\ \text{apply}(\square, B, C) \quad \text{apply}(\square, B', C') \end{array}$$

$$\text{apply}(\square, \begin{array}{c} \textcircled{x} \\ \swarrow \quad \searrow \\ B \quad B' \end{array}, C) = \begin{array}{c} \textcircled{x} \\ \swarrow \quad \searrow \\ \text{apply}(\square, B, C) \quad \text{apply}(\square, B', C) \end{array}$$

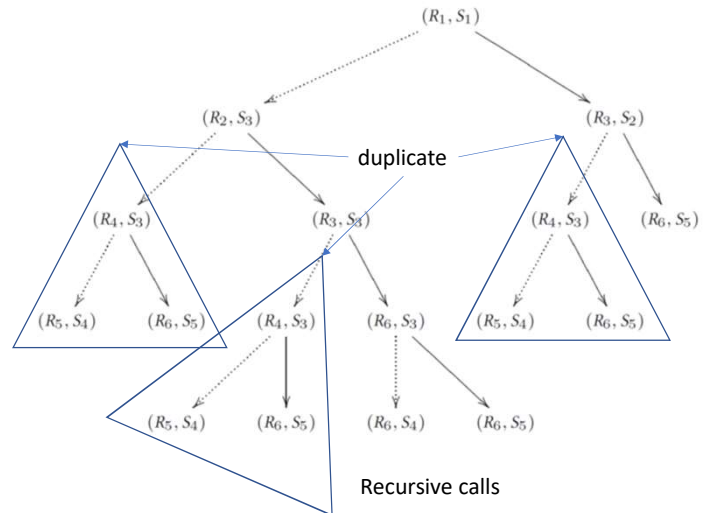
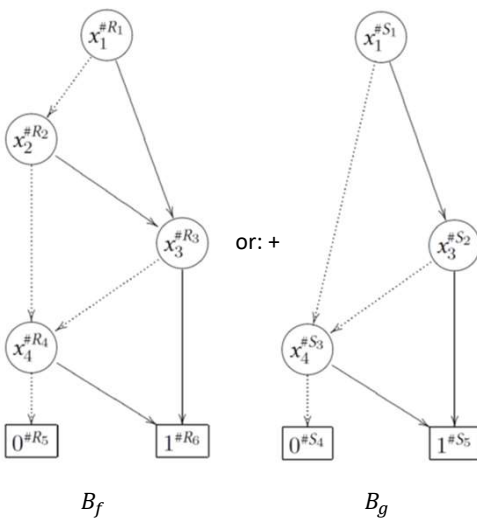
C is a terminal node or
non-terminal with $\text{var}(\text{root}(C)) > x$

$$\text{apply}(\square, B, \begin{array}{c} \textcircled{x} \\ \swarrow \quad \searrow \\ C \quad C' \end{array}) = \begin{array}{c} \textcircled{x} \\ \swarrow \quad \searrow \\ \text{apply}(\square, B, C) \quad \text{apply}(\square, B, C') \end{array}$$

B is a terminal node or
non-terminal with $\text{var}(\text{root}(B)) > x$

$$\text{apply}(\square, \boxed{u}, \boxed{v}) = \boxed{u \square v}$$

Example: Apply



The recursive apply implementation will generate an OBDD.

- Apply reduce to convert it back to a ROBDD

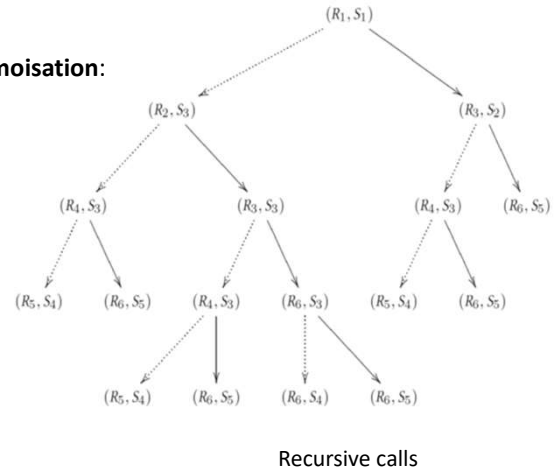
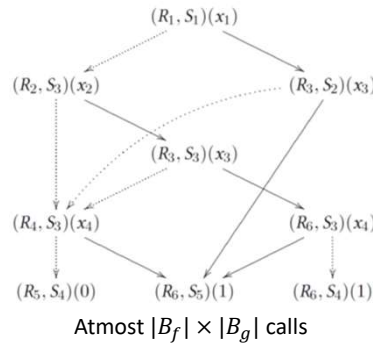
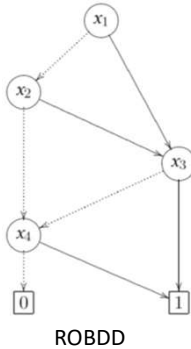
Example: Apply

In the tree of recursive calls, there are many calls to apply with the same arguments

- Each invocation of apply where at least one of the arguments is non-terminal generates two further calls to apply: the number of calls is worst-case exponential in the sizes of the original diagrams

Reason: We are not taking into account the **sharing** in BDDs.

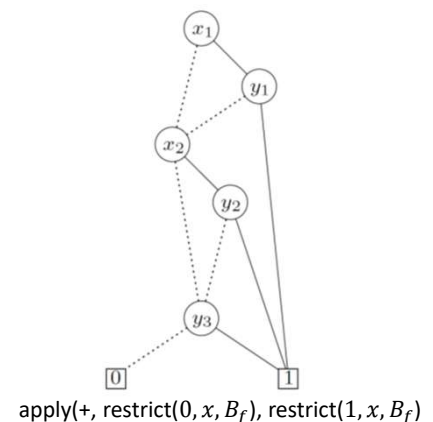
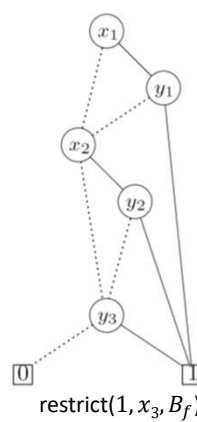
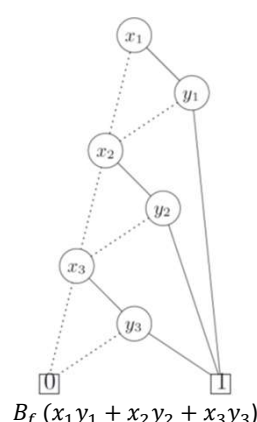
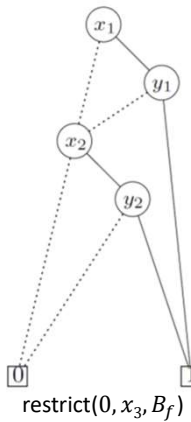
Solution: We can greatly improve the run-time by using **memoisation**: remembering the results of previous calls



Other Operations

$\text{restrict}(0, x, B_f)$ computes ROBDD for $f_{x=0}$

- For each node n labeled with x , incoming edges are redirected to $\text{lo}(n)$, and the node n is removed
- Resulting BDD is then reduced with reduce



$\text{exists}(x, B_f)$ computes ROBDD for $\exists x. f$ ($\exists x. f \equiv f_{x=0} + f_{x=1}$)

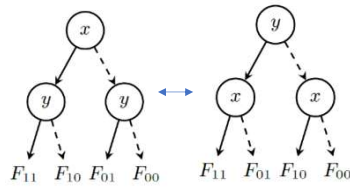
Implemented as $\text{apply}(+, \text{restrict}(0, x, B_f), \text{restrict}(1, x, B_f))$

Q. How to find $\forall x. f$? A: $f_{x=0} f_{x=1}$

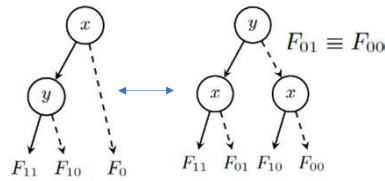
Dynamic Variable Ordering

Dynamic ordering

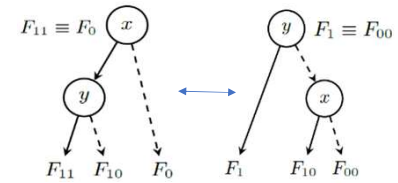
- Starts with user-provided static order
- Key observation [Friedman]: Given ROBDD with $x_1 < \dots < x_i < x_{i+1} < \dots < x_n$,
 - Permuting $x_1 \dots x_i$ does not affect ROBDD nodes labeled by $x_{i+1} \dots x_n$
 - Permuting $x_{i+1} \dots x_n$ does not affect ROBDD nodes labeled by $x_1 \dots x_i$
 - Variables in adjacent levels are easily swappable



Variable swap between x and y



Variable swap resulting more nodes



Variable swap resulting same #nodes